From a loop to a workflow

A field guide to the runnable QC exercise in PurdueNfWorkshop.

01 / THE IDEA

Describe dependencies; let Nextflow schedule the work

A workflow manager connects the steps of an analysis through declared inputs and outputs. Nextflow can run independent tasks in parallel and reuse completed work after an interruption.



FASTP JSON reports and FASTQC ZIP reports both feed MultiQC. Nextflow passes process outputs directly; downstream tasks do not read the published results folder.

Nextflow uses a Groovy-based DSL and channels; Snakemake uses Python-based rules and input/output files. Both support HPC execution and containers.

02 / VOCABULARY

Five ideas to recognize

Term	Meaning in this exercise
Process	A tool step: FASTP, FASTQC, or MULTIQC.
Task	One execution of a process for its input.
Channel	A stream of files or values passed between processes.
Workflow	The block that connects the processes and channels.
Executor	Where tasks run: within one allocation (local) or as separate Slurm jobs.

03 / SETUP

Download the training inputs to scratch

```
git clone https://github.com/zhan4429/PurdueNfWorkshop.git "$RCAC_SCRATCH/nextflow_workflow"
cd "$RCAC_SCRATCH/nextflow_workflow"
bash setup.sh
```

Already have the repository? Enter that folder and run `bash setup.sh`. Inputs always go to `$RCAC_SCRATCH/nextflow_workshop/fastq`. Existing valid files are kept; failed transfers stop the script.

> Scratch is temporary and is not backed up. Copy checked results and provenance to durable project storage before cleanup.

04 / INPUT MODEL

11 files are not 11 biological samples

Setup downloads seven R1 and four R2 files. This teaching example processes **each file independently in single-end mode**. Each route creates 11 FASTP tasks, 11 FASTQC tasks, and one MULTIQC task: **23 tasks**.

Independently filtered mates are not synchronized paired-end outputs. For a paired-end analysis, group mates and run fastp with `-i/-I/-o/-O` together.

Build the QC workflow

The code fragments below come from main.nf; use the complete file to run.

05 / FROM LOOP TO PIPELINE

The same analysis, three execution routes

Bash loop	Nextflow
One input file at a time	Independent tasks can run concurrently
Resubmission starts from the first file	`-resume` reuses matching completed tasks
One Slurm allocation	Local executor: one allocation; Slurm executor: child jobs

06 / ONE PROCESS

FASTP: stage a file, run a tool, emit outputs

```
process FASTP {
    tag "${reads.simpleName}"
    module 'biocontainers:fastp'
    cpus 2
    memory '4 GB'
    time '30m'
    publishDir "${params.outdir}/fastp", mode: 'copy'

    input:
    path reads

    output:
    path '*.trimmed.fastq.gz', emit: trimmed
    path '*.fastp.json',       emit: json
    path '*.fastp.html'

    script:
    """
    fastp -w ${task.cpus} -i "${reads.name}" \\
        -o "${reads.simpleName}.trimmed.fastq.gz" \\
        -j "${reads.simpleName}.fastp.json" \\
        -h "${reads.simpleName}.fastp.html"
    """
}
```

`path` stages files; `emit` names an output channel; `publishDir` copies declared outputs. Parameters and task resources are defined directly in `main.nf`.

07 / DATAFLOW

Connect the steps without a shell loop

```
workflow {
    reads_ch = channel.fromPath("${params.fastq_dir}/*.fastq.gz", checkIfExists: true)
    FASTP(reads_ch)
    FASTQC(FASTP.out.trimmed)
    MULTIQC(FASTP.out.json.mix(FASTQC.out.zip).collect(sort: true))
}
```

`mix()` combines the report streams; `collect(sort: true)` waits for all reports and puts them in a stable order. MultiQC runs once after all required reports are ready.

> **Exercise:** If a FASTP task fails, the run fails by default. Resubmit the same launcher: matching completed tasks can be cached; failed or missing tasks run again.

Run on HPC

Always launch the Nextflow manager on a compute node using sbatch.

08 / ENVIRONMENT

Load modules directly in each job script

```
# Nextflow launchers:
module load biocontainers nextflow
# Bash loop:
module load biocontainers fastp fastqc multiqc
```

Each script loads its own modules. The Nextflow processes load the required tool with their `module` directive. No shared environment helper or settings file is needed.

Use `module avail` or `module spider` to check available tools, and `slist` to find your allocation.

09 / CONFIGURATION

Local by default; Slurm with hpc.config

`main.nf` defines the shared FASTQ path and output directory. Each process declares 4 GB of memory and a 30-minute task limit. The local launcher uses Nextflow's default local executor. Set the job's allocation with its `#SBATCH` directives.

The HPC launcher adds `hpc.config` to submit tasks as separate Slurm jobs:

```
process {
  executor = 'slurm'
  queue = 'cpu'
  clusterOptions = '-A YOUR_ACCOUNT'
}
```

Before running, replace `YOUR_ACCOUNT` in this config. `queue = 'cpu'` sets the task partition; change it if needed. These settings configure child jobs independently of the head job's `sbatch` options.

10 / SUBMIT AND MONITOR

Submit from $RCAC_SCRATCH/nextflow_workflow

```
sbatch -A YOUR_ACCOUNT -p YOUR_PARTITION qc_forloop.sh
sbatch -A YOUR_ACCOUNT -p YOUR_PARTITION qc_nextflow_local.sh
sbatch -A YOUR_ACCOUNT -p YOUR_PARTITION qc_nextflow_hpc.sh
squeue -u "$USER"
```

Replace the submission placeholders and the account placeholder in `hpc.config`. Run the examples in turn. Do not overlap submissions of the same Nextflow launcher.

> The 30-minute head-job limit includes waiting for child jobs. Resubmit the same launcher to resume after an interruption.

Resume. Debug. Preserve.

Use the same launcher, input files, configuration, and work tree.

11 / WHERE FILES LIVE

One working directory, separate result folders

Location (relative paths start in nextflow_workflow/)	Contents
`$RCAC_SCRATCH/nextflow_workshop/fastq`	11 downloaded training inputs
`results/{forloop,local,slurm}/`	Published fastp, fastqc, and multiqc outputs
`.nextflow/cache`	Task metadata needed to resume
`work/{local,slurm}/`	Task scripts, logs, and outputs needed to resume

Final reports: `results/<route>/multiqc/multiqc_report.html`. This exercise uses Purdue modules; it does not configure Nextflow-managed Apptainer images.

12 / RESUME AND INSPECT

Resubmit the same route; retain cache and work

```
# From the checkout root; both launchers already include -resume:
sbatch -A YOUR_ACCOUNT -p YOUR_PARTITION qc_nextflow_hpc.sh
# After completion, load Nextflow, then:
cd "$RCAC_SCRATCH/nextflow_workflow"
nextflow log
nextflow log last -f name,status,exit,workdir
```

Read the job log in the checkout root and `.nextflow.log` in the launch folder. In a failed task's reported work directory, read `.command.sh`, `.command.err`, and `.exitcode`. Debug tool commands in an interactive compute allocation.

Run one Nextflow example at a time. To resume an older run after switching examples, use `-resume RUN_NAME` from `nextflow log`. Changed inputs, scripts, or software can invalidate cached tasks.

13 / COMMAND SYNTAX

One dash for Nextflow options; two for parameters

`-resume`, `-c`, `-profile`, and `-r` are Nextflow run options. `--fastq_dir` and `--outdir` are parameters defined by this pipeline.

`--resume` is not the resume option. The supplied launchers preserve the correct executor, parameters, launch directory, and work directory on every resubmission.

14 / SAVE BEFORE CLEANUP

Check reports and preserve durable copies

```
cd "$RCAC_SCRATCH/nextflow_workflow"
ls results/slurm/multiqc/multiqc_report.html
myquota
purgelist
# Save results and provenance to durable storage BEFORE cleanup.
nextflow log
nextflow clean RUN_NAME -n  # replace RUN_NAME; inspect preview
# Only after checking: nextflow clean RUN_NAME -f
```

Cleanup prevents affected tasks from resuming. Never delete the whole checkout as a shortcut. For nf-core pipelines, follow the nf-core usage guide for pipeline inputs and software configuration. Full workshop instructions: `README.md`.